

Drone Distance Document

July 2026

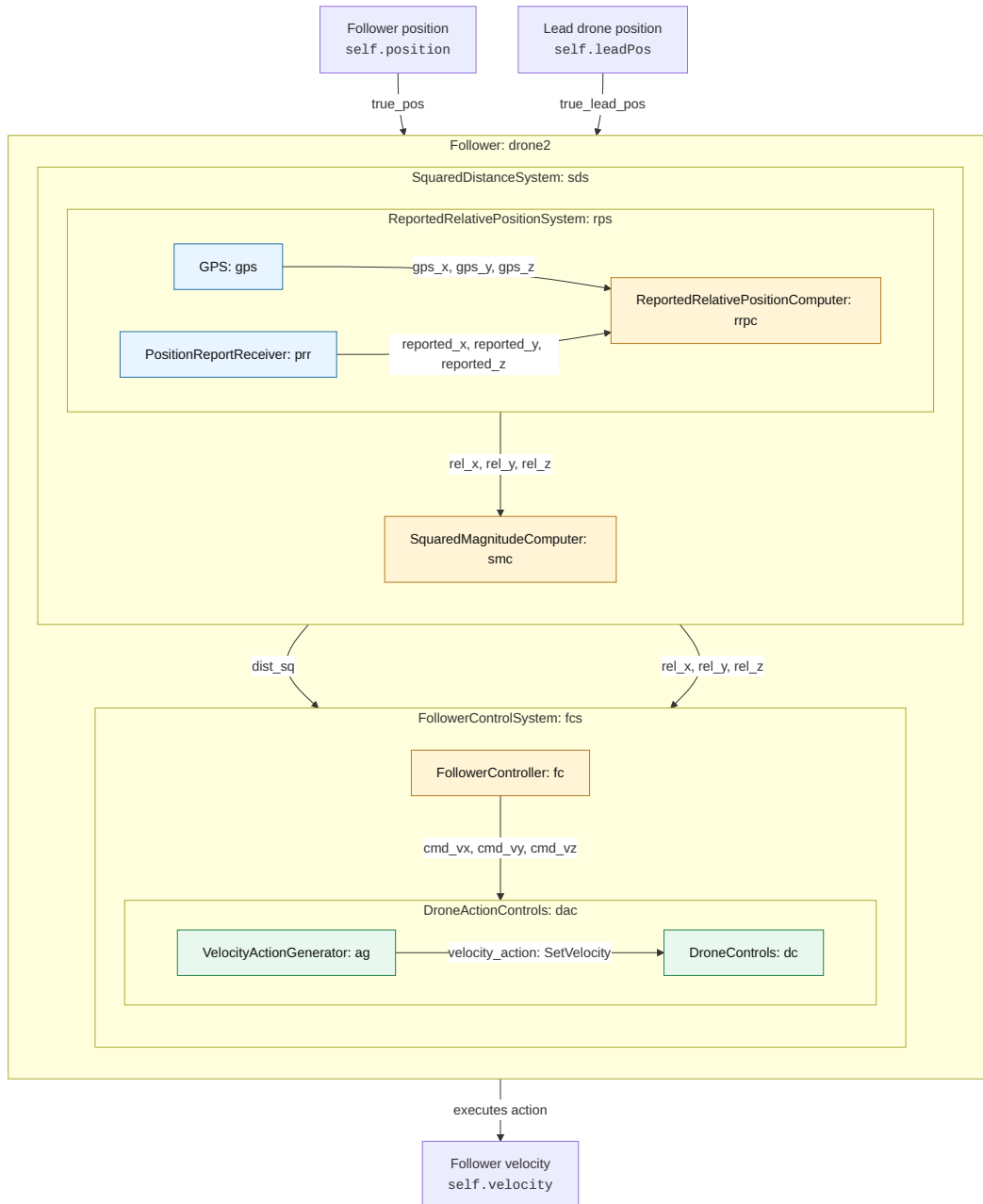


Figure 1: Composition and data flow of the follower implementation.

1 Idea

The contract describes a scenario with two quadcopter drones, Leader and Follower. The Leader drone behavior is taken from the patrol example in `followPatrol.scenic`. The Follower drone behavior is initialized there with the `Follow` behavior, but then overridden by a compositional implementation in `drone_distance.contract`.

The Leader does the Patrol behavior, which is just going around the environment slowly. The Follower starts 100 units away, and must get close to the Leader, but not too close, and continue to stay within a certain range as best as possible.

This corresponds to four verified end-to-end results: a progress property, a simulation-tested liveness property, directional movement safety, and a never-crash invariant with a positive separation floor.

The minimum and maximum follow distances are 5 and 15. The approach gain is 0.2, the retreat gain is 0.5, the maximum velocity in each component is 3, the maximum Leader speed is 2, and the simulation timestep is 0.1 seconds.

2 Components

2.1 GPS

Sensors.

- `self.position`, exposed internally as `true_pos` (`Vector`)

Inputs. None.

Outputs.

- `gps_x`, `gps_y`, `gps_z` (`float`)

Description. This component reports the exact position of the Follower.

2.2 PositionReportReceiver

Sensors.

- `self.leadPos`, exposed internally as `true_lead_pos` (`Vector`)

Inputs. None.

Outputs.

- `reported_x`, `reported_y`, `reported_z` (`float`)

Description. This component receives and outputs the (exact) position as “reported” by the Leader. Together with the GPS component, it models ideal sensing: positions are available without noise, error, or delay.

2.3 ReportedRelativePositionComputer

Inputs.

- `own_x`, `own_y`, `own_z` (`float`)
- `reported_x`, `reported_y`, `reported_z` (`float`)

Outputs.

- `rel_x`, `rel_y`, `rel_z` (`float`)

Description. This component computes the relative position of the Leader from the Follower. Given that the Leader is at position $\hat{r}_\ell$ and the Follower is at position $\hat{r}_f$, it returns $\hat{r}_\ell - \hat{r}_f$.

2.4 ReportedRelativePositionSystem

Inputs. None.

Outputs.

- `rel_x`, `rel_y`, `rel_z` (float)

Composition. See Figure 1.

- `gps`: GPS
- `prr`: PositionReportReceiver
- `rrpc`: ReportedRelativePositionComputer

The subcomponents are connected as follows:

- `gps.gps_x`, `gps.gps_y`, and `gps.gps_z` connect to `rrpc.own_x`, `rrpc.own_y`, and `rrpc.own_z`, respectively.
- `prr.reported_x`, `prr.reported_y`, and `prr.reported_z` connect to `rrpc.reported_x`, `rrpc.reported_y`, and `rrpc.reported_z`, respectively.
- `rrpc.rel_x`, `rrpc.rel_y`, and `rrpc.rel_z` connect to the corresponding outputs of the system.

Description. This component gets the Follower's position and the Leader's position from its internal sensor components and asks the `ReportedRelativePositionComputer` for the relative position vector, then outputs that vector.

2.5 SquaredMagnitudeComputer

Inputs.

- `rel_x`, `rel_y`, `rel_z` (float)

Outputs.

- `dist_sq` (float)

Description. This component computes the squared magnitude of a vector.

2.6 SquaredDistanceSystem

Inputs. None.

Outputs.

- `dist_sq` (float)
- `rel_x`, `rel_y`, `rel_z` (float)

Composition. See Figure 1.

- `rps`: ReportedRelativePositionSystem
- `smc`: SquaredMagnitudeComputer

The subcomponents are connected as follows:

- `rps.rel_x`, `rps.rel_y`, and `rps.rel_z` connect both to the corresponding outputs of the system and to `smc.rel_x`, `smc.rel_y`, and `smc.rel_z`, respectively.
- `smc.dist_sq` connects to the `dist_sq` output of the system.

Description. This component combines the two preceding systems. It gets the relative position from the former and asks the latter for the squared magnitude of that vector, then outputs *both* the vector and its squared magnitude.

2.7 FollowerController

Parameters.

- `min_dist`, `max_dist`, `gain`, `retreat_gain`, `max_component_speed`

Inputs.

- `dist_sq` (float)
- `rel_x`, `rel_y`, `rel_z` (float)

Outputs.

- `cmd_vx`, `cmd_vy`, `cmd_vz` (float)

Description. This component takes as input the relative position vector and its squared magnitude, then uses the information to decide with what velocity the Follower should move.

The decision depends on the (squared) distance to the Leader. The component takes as parameters a minimum and maximum distance which determine a spherical shell (a “band”) centered around the Leader. If the distance from the Follower to the Leader is greater than the outer radius of the band, the Follower moves closer. If it is less than the inner radius, the Follower retreats. At either boundary or within the band, the Follower hovers.¹

The magnitude² of the commanded velocity depends on the component-wise distance between the two drones and three parameters: `max_component_speed`, `gain`, and `retreat_gain`.

The first parameter, `max_component_speed`, ensures that the Follower is never commanded to move too fast. Write $M = \text{max_component_speed}$. Each component v is clamped to the interval $[-M, M]$ using $\min(\max(v, -M), M)$. Thus the commanded speed is never greater than $M\sqrt{3}$. The parameter `gain` controls the proportional approach response outside the outer boundary. The separate `retreat_gain` controls the proportional retreat response inside the inner boundary.

2.8 VelocityActionGenerator

Inputs.

- `cmd_vx`, `cmd_vy`, `cmd_vz` (float)

Outputs.

- `velocity_action` (SetVelocity)

Description. This component takes in a commanded velocity and turns it into a `SetVelocity`, which is an AirSim-specific way of telling a drone to start moving with a certain velocity.

2.9 DroneControls

Inputs. None.

¹Hovering was broken for a while: whenever the controller commanded a velocity of zero, the Follower would sink to the ground and only recover after hitting it. The hover logic was actually fine—the bug was in the conversion from Scenic velocities to AirSim. We were routing velocities through the *location* converters, which apply the world offset and a $\times 100$ mesh scaling, so with `worldOffset` = (0,0,50) a commanded (0,0,0) came out as (0,0,0.5): a constant 0.5 m/s downward push every time the Follower was told to stop. And because the location path never flips the z -axis (Scenic is z -up, AirSim’s NED frame is z -down), corrective climbs were delivered as further descent, so the drone couldn’t recover until the ground stopped it. The fix was to give velocities their own axis-only converters (no offset, no scaling, correct sign), rather than to special-case the hover.

²Strictly speaking the direction too: because we clamp each coordinate independently rather than scaling the vector uniformly (this just makes the Lean/math easier), the clamped velocity need not point exactly along $\hat{r}_\ell - \hat{r}_f$. What survives is that each component keeps its sign, so the motion still points into the correct octant—toward the Leader when approaching, away when retreating. This is what the directional-safety contract establishes.

Outputs. None.

Actions.

- `velocity_action` (`SetVelocity`)

Description. This component receives the action produced by the `VelocityActionGenerator` and “actually” executes it.

2.10 DroneActionControls

Inputs.

- `cmd_vx`, `cmd_vy`, `cmd_vz` (float)

Outputs. None.

Composition. See Figure 1.

- `ag`: `VelocityActionGenerator`
- `dc`: `DroneControls`

The subcomponents are connected as follows:

- The system inputs `cmd_vx`, `cmd_vy`, and `cmd_vz` connect to the corresponding inputs of `ag`.
- `ag.velocity_action` connects to `dc.velocity_action`.

Description. This component composes the two subcomponents listed above. It receives the commanded velocity from the controller, passes it to the action generator, and then passes the resulting action to the drone controls, which execute it.

2.11 FollowerControlSystem

Parameters.

- `min_dist`, `max_dist`, `gain`, `retreat_gain`, `max_component_speed`

Inputs.

- `dist_sq` (float)
- `rel_x`, `rel_y`, `rel_z` (float)

Outputs. None.

Composition. See Figure 1.

- `fc`: `FollowerController`, instantiated with the five parameters listed above
- `dac`: `DroneActionControls`

The subcomponents are connected as follows:

- The system inputs `dist_sq`, `rel_x`, `rel_y`, and `rel_z` connect to the corresponding inputs of `fc`.
- `fc.cmd_vx`, `fc.cmd_vy`, and `fc.cmd_vz` connect to `dac.cmd_vx`, `dac.cmd_vy`, and `dac.cmd_vz`, respectively.

Description. This component takes the relative position vector and its squared magnitude and passes them to the `FollowerController`, which uses them to compute the commanded velocity. It then passes that velocity to the `DroneActionControls`, which turns it into an action and executes it.

2.12 Follower

Parameters.

- `min_dist`, `max_dist`, `gain`, `retreat_gain`, `max_component_speed` (float)

Inputs. None.

Outputs. None.

Composition. See Figure 1.

- `sds`: `SquaredDistanceSystem`
- `fcs`: `FollowerControlSystem`, instantiated with the five parameters listed above

The subcomponents are connected as follows:

- `sds.dist_sq` connects to `fcs.dist_sq`.
- `sds.rel_x`, `sds.rel_y`, and `sds.rel_z` connect to `fcs.rel_x`, `fcs.rel_y`, and `fcs.rel_z`, respectively.

Description. This is the whole Follower drone! It is made up of the `SquaredDistanceSystem` which is like the perception system from the car example and the `FollowerControlSystem` which is a composite of the control system and the actuator from the car example.

3 Contracts

3.1 Position and Distance Contracts

3.1.1 GPSAccurate

Kind. Assumption on the GPS component.

Description. This is a simple assumption that the “GPS” component is accurate; i.e. the position it reports is exactly equal to the actual position of the Follower.

3.1.2 PositionReportReceiverAccurate

Kind. Assumption on the `PositionReportReceiver` component.

Description. This is the analogous assumption on the Leader’s “GPS” and reporting: it codifies the assumption that the Leader knows its own position accurately and reports it to the Follower accurately.

3.1.3 ReportedRelativePositionComputerSemantics

Kind. Proved directly about the `ReportedRelativePositionComputer` component.

Description. This is a contract that states the relative position is computed correctly as the component-wise difference of the Leader’s position and the Follower’s position.

3.1.4 ReportedRelativePositionAccurate

Kind. Refinement of the contracts for the GPS, position report receiver, and relative position computer composed over the `ReportedRelativePositionSystem`.

Description. This is a refinement of the composite of the above three contracts: it states that the computed relative position is equal to the difference of the actual positions of the two drones, not just the reported positions.

3.1.5 SquaredMagnitudeComputerSemantics

Kind. Proved directly about the `SquaredMagnitudeComputer` component.

Description. This states that the squared magnitude of the relative position (or really whatever vector the `SquaredMagnitudeComputer` receives) is in fact the squared magnitude of that vector.

3.1.6 SquaredDistanceSystemAccuracy

Kind. Refinement over the `SquaredDistanceSystem` using the two contracts above.

Description. This is a refinement of the above two which combines the two guarantees exactly into one contract.

3.2 Controller and Actuation Contracts

3.2.1 FollowerControllerSemantics

Kind. Proved directly about the `FollowerController` component.

Description. Under sensible assumptions on the parameters (for example, both gains are positive and the band is not inverted), this contract states the following:

- Whenever the Follower is inside the band, its commanded velocity is zero.
- Outside the band, the commanded velocity equals the component's clamped proportional response: `gain` for approach and `retreat_gain` for retreat.

Assumptions. Both gains and the maximum component speed are positive, $0 \leq \text{min_dist} \leq \text{max_dist}$.

3.2.2 FollowerControllerDirectionalSafety

Kind. Refinement of `FollowerControllerSemantics`.

Description. This is a refinement of the above into a strictly weaker but still necessary guarantee on the Follower's behavior. It states that the follower is always pointed in the "right" direction, i.e. if it is too far away, the dot product of the commanded velocity and position vectors is positive (so it moves towards the Leader) and if it is too close the dot product is negative (so it moves away).

This contract also states (componentwise) the $\sqrt{3}M$ bound on the commanded velocity.

3.2.3 VelocityActuation

Kind. Assumption on the `DroneActionControls` component, with correctness 0.99 and confidence 0.999.

Description. We assume that the velocity actuator provided by AirSim works as intended with the stated correctness and confidence.

3.2.4 FollowerVelocityDirectionalSafety

Kind. Refinement over the `FollowerControlSystem` using the directional controller and actuation contracts above.

Description. This is a refinement over the previous contract which makes the same claims about the *actual* next velocity of the drone, rather than about the commanded velocity (which we assumed with small probability may not have been executed correctly).

3.2.5 FollowerVelocitySemantics

Kind. Refinement over the `FollowerControlSystem` using the exact controller semantics and actuation contracts above.

Description. This does the same as the above but for the exact equality claims in `FollowerControllerSemantics`.

In particular, the next velocity is zero inside the band, uses `gain` in the approach branch, and uses `retreat_gain` in the retreat branch.

3.3 End-to-End Contracts

3.3.1 KeepsFollowDistanceProgress

Kind. Refinement of `FollowerVelocitySemantics` and `SquaredDistanceSystemAccuracy` composed over the complete Follower.

Description. This contract says that if the Follower is too far away, it is guaranteed to get closer to the Leader in the next step.

Assumptions. The parameters are positive, position follows the velocity integration law with timestep 0.1, and the Leader moves at most 0.2 units per step. Above the outer boundary, the Follower’s minimum approach speed is greater than the Leader’s maximum speed.

3.3.2 KeepsFollowDistanceLiveness

Kind. Tested directly on the complete Follower using simulation.

Description. This is the tested half of the liveness guarantee, which says that the Follower eventually gets inside the outer ring of the band around the Leader.

The contract is tested on the complete Follower in the AirSim patrol scenario. Each simulation runs for at most 600 steps. Testing uses confidence 0.999, a batch size of 1, NumPy seed 42, and a five-minute testing time budget.

3.3.3 FollowerMovementDirectionalSafety

Kind. Refinement of `FollowerVelocityDirectionalSafety` and `SquaredDistanceSystemAccuracy` composed over the complete Follower.

Description. We prove that the Follower’s actual one-step displacement always points in the right direction: too far implies the displacement’s dot product with the true relative position is non-negative (moving toward the Leader), too close implies it is non-positive (moving away). This is directional only — it constrains the sign of one step, not the distance over time, so it does not by itself rule out a collision.

Assumptions. The follow-distance band is not inverted, and position follows the velocity integration law with the configured timestep of 0.1.

3.3.4 NeverCrash

Kind. Refinement over the complete Follower. It composes `FollowerVelocitySemantics`, which gives the exact velocity semantics, with `SquaredDistanceSystemAccuracy`.

Description. This contract states that the Follower never crashes into the Leader. More precisely, the true distance always stays above a positive floor. It assumes that the Leader cannot travel the inner-band radius in one time step and that an approach step from outside the spherical shell cannot overshoot that floor. The guarantee is

$$\text{always } \text{true_dist_sq} \geq R^2, \quad R = \text{min_dist} - \text{max_leader_speed} \cdot \text{timestep}.$$

Assumptions. The parameters and collision floor are positive. The Follower starts at or above the floor, position follows the velocity integration law, and the Leader's displacement is bounded by its maximum speed. Throughout the retreat region, the retreat speed is at least the Leader's maximum speed and the velocity clamp is inactive. An approach step from beyond the outer boundary cannot cross the floor.

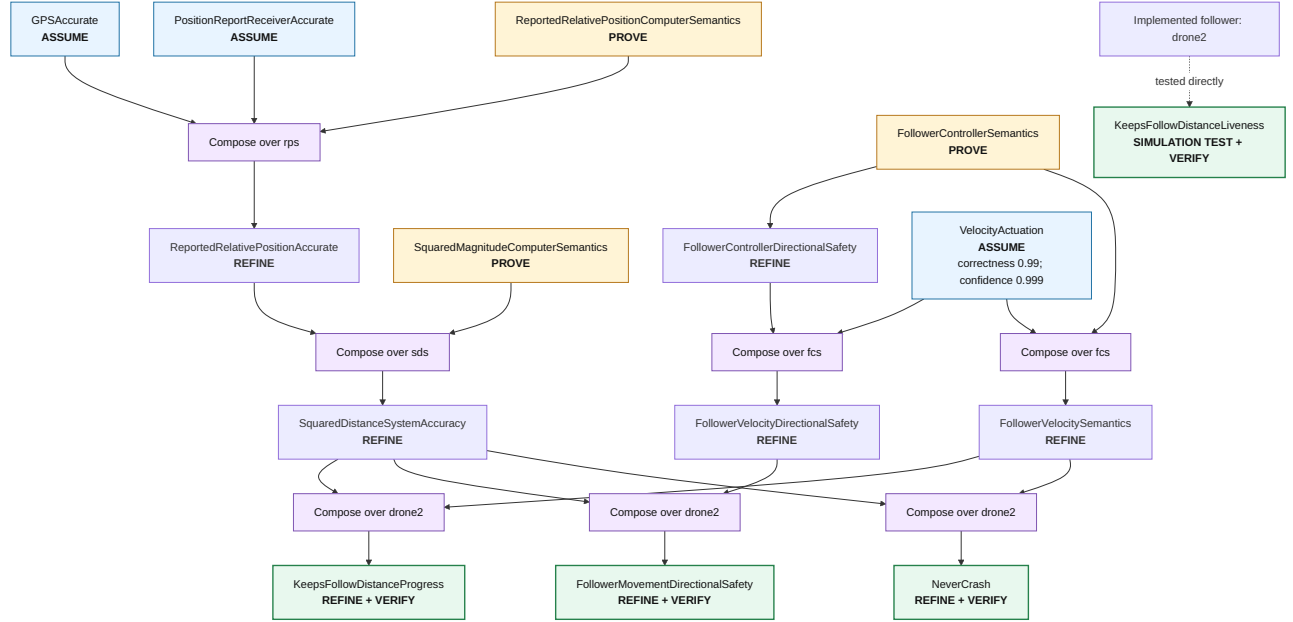


Figure 2: Contract composition, proof, refinement, and verification structure.

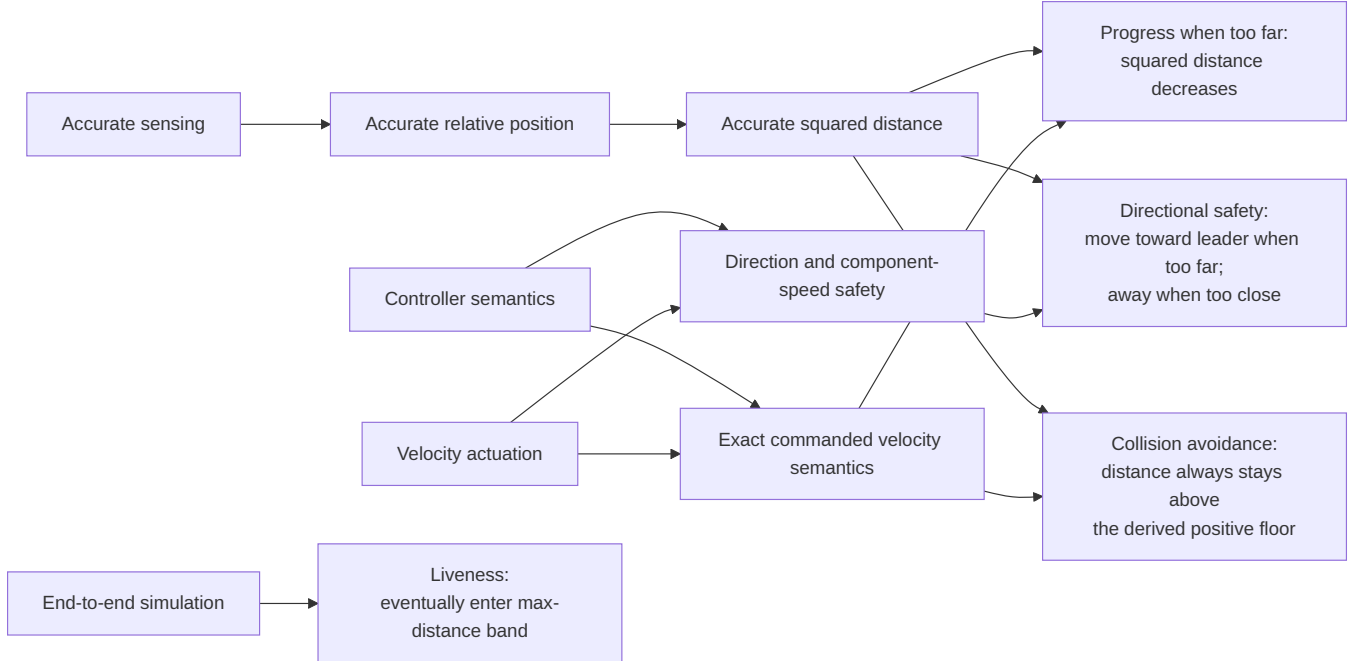


Figure 3: Summary of the properties established by the contracts.